

4.Implement RSA Algorithm for Secure Data Transmission using Public and Private Keys (Google Colab)

1. Description

RSA (Rivest-Shamir-Adleman) is one of the most widely used **asymmetric cryptographic algorithms** for secure communication over insecure networks. Unlike symmetric encryption, RSA uses **two different keys**:

- **Public Key** – Used to encrypt data.
- **Private Key** – Used to decrypt data.

The RSA algorithm is based on the mathematical difficulty of factoring very large prime numbers. It provides confidentiality, authentication, and secure key exchange, making it suitable for secure communication, digital signatures, SSL/TLS, online banking, and secure email systems.

In this practical, students will implement RSA encryption and decryption using Python in Google Colab. The program will generate RSA key pairs, encrypt a plaintext message using the public key, and decrypt it using the private key.

2. Objectives

After completing this practical, students will be able to:

- Understand the concept of asymmetric encryption.
- Learn the working principle of the RSA algorithm.
- Generate RSA public and private keys.
- Encrypt plaintext using the public key.
- Decrypt ciphertext using the private key.
- Understand secure data transmission using public-key cryptography.
- Implement RSA encryption in Python using Google Colab.

3. Learning Outcomes

After successful completion of this experiment, students will be able to:

- Explain the difference between symmetric and asymmetric encryption.
- Understand RSA key generation.
- Perform encryption and decryption using RSA.
- Analyze how public and private keys work together.
- Apply RSA in real-world secure communication systems.
- Develop secure Python applications using RSA.

4. Software Requirements

Requirement	Details
-------------	---------

Platform	Google Colab
Programming Language	Python 3.x
Library	cryptography
Internet Connection	Required (for first-time installation)
Operating System	Windows/Linux/macOS

5. Installation (Google Colab)

Run the following command only once.

```
!pip install cryptography
```

Import the required modules.

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
```

```
from cryptography.hazmat.primitives import serialization, hashes
```

6. Theory

What is RSA?

RSA is an asymmetric encryption algorithm developed by

- Ron Rivest
- Adi Shamir
- Leonard Adleman

in 1977.

Unlike symmetric encryption, RSA uses two keys.

Public Key

Can be shared with everyone.

Used for encryption.

Private Key

Kept secret.

Used for decryption.

RSA Working

Step 1

Generate two large prime numbers

p
q

Step 2

Compute

$$n = p \times q$$

Step 3

Compute Euler's Totient

$$\phi(n) = (p - 1)(q - 1)$$

Step 4

Choose

e

such that

$$1 < e < \phi(n)$$

and

$$\gcd(e, \phi(n)) = 1$$

Step 5

Compute

d

such that

$$(d \times e) \bmod \phi(n) = 1$$

Public Key

(e,n)

Private Key

(d,n)

Encryption

$$\text{Cipher} = \text{Message}^e \bmod n$$

Decryption

$$\text{Message} = \text{Cipher}^d \bmod n$$

Advantages

- Highly secure
- Public key distribution
- Digital signatures
- Authentication
- Secure key exchange

Applications

- HTTPS
- SSL/TLS

- Banking
- Digital Certificates
- Email Security
- VPN
- Blockchain

7. Python Program (Google Colab with Detailed Comments)

```
# =====
# Practical: RSA Asymmetric Encryption
# Platform : Google Colab
# =====

# Install library (Run only first time in Colab)
# !pip install cryptography

# Import required modules
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import serialization, hashes

# -----
# STEP 1 : Generate RSA Private Key
# -----
private_key = rsa.generate_private_key(
    public_exponent=65537, # Standard public exponent
    key_size=2048         # RSA Key Size
)

# -----
# STEP 2 : Generate Public Key
# -----
public_key = private_key.public_key()

# -----
# STEP 3 : Display Public Key
# -----
public_pem = public_key.public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo
)
```

```

print("===== PUBLIC KEY =====")
print(public_pem.decode())

# -----
# STEP 4 : Display Private Key
# -----
private_pem = private_key.private_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PrivateFormat.TraditionalOpenSSL,
    encryption_algorithm=serialization.NoEncryption()
)

print("\n===== PRIVATE KEY =====")
print(private_pem.decode())

# -----
# STEP 5 : Read Plain Text
# -----
message = input("Enter Message : ")

print("\nOriginal Message :", message)

# -----
# STEP 6 : Encrypt Message
# -----
ciphertext = public_key.encrypt(
    message.encode(),
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

```

```

print("\nEncrypted Message (Cipher Text):")
print(ciphertext)

# -----
# STEP 7 : Decrypt Message
# -----

plaintext = private_key.decrypt(

    ciphertext,

    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

print("\nDecrypted Message :")
print(plaintext.decode())

# -----
# STEP 8 : Verification
# -----

if message == plaintext.decode():
    print("\nRSA Encryption and Decryption Successful.")
else:
    print("\nVerification Failed.")

```

8. Output

```

===== PUBLIC KEY =====
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAlj2Di1nznx1rlVoTW/CX
K2UqHttwYqS/Iq9Wa+B8GNSGbdDyVe1ctLXC/W1lYUobglTN/kUBJQgVOJarSP6i
kq5z6Mah5bcY8b56e2f5w2w38b5gKZkMaDpKZpB4WoopJxQngesdeDQ2XwWarzt8
413nWo7q2C6aN0Sb8UFFy8yT6Sf5Zo/2rJm28m0fhaIp7sBqxlrrqNihh91/dOVZ
tTCSgmeSpIl6HY1uZ6/fvZuBS0JGiAdXg6tg3SESnqQiBApbP81C1cOeME12IQI

```

br+Rhl+7sm7AFqXfnhoWPtP9hTPa4WaSBm4nwIgBUqE30CuyhNyXFTs1OwM+z7BV
4QIDAQAB

-----END PUBLIC KEY-----

===== PRIVATE KEY =====

-----BEGIN RSA PRIVATE KEY-----

MIIEpAIBAAKCAQEAJ2Di1nznx1rlVoTW/CXK2UqHttwYqS/Iq9Wa+B8GNSGbdDy
Ve1ctLXC/W1lYUobglTN/kUBJQgVOJarSP6ikq5z6Mah5bcY8b56e2f5w2w38b5g
KZkMaDpKZpB4WoopJxQngesdeDQ2XwWarzt8413nWo7q2C6aN0Sb8UFFy8yT6Sf5
Zo/2rJm28m0fhaIp7sBqxlrqNihh91/dOVZtTCSgmeSpll6HY1uZ6/fvZuBS0J
GiAdXg6tg3SESnnQqIBApbP81C1cOeME12IQlbr+Rhl+7sm7AFqXfnhoWPtP9hTPa
4WaSBm4nwIgBUqE30CuyhNyXFTs1OwM+z7BV4QIDAQABAoIBAAMtbV/WdbvR59Gg
9en0bAOmFqHQlvx/GGJcjw+y6XguXhWqY3mt20+G5mCtCxKBIMhQNVdiYjjbhsFe
rf3NXtuKnRc/xJkV/U0D9rvIUAPqGo7+QP/lBZlnUP81Nn2c6h//G3llfNPRcBK9
5lnaGi45cSUET6yyfZ7GHXGY7EFryHJ2P++m+MiEgyB6a3vd23R5Wdne+HiKANQU
7otqj4r0svC+/kopzX8DWnAUZRUPYyIGk6kf0vEs4W7TXhTMg/wrWIZtFgp/qsko
IQu0pfox5LsPT9mSYVhnc4lH3B7z3Um2SulczYOp7Q2SKSPtR+7YZ/YfklJziscM
yKq29vECgYEA01aM40JP0lkMjJuTZ18DEHj94+iRzK6wyckSCQS6flMQj6Jd4wt4
cd3o8ULBUFC6IPFYZbq6HL7E1cFCGeuFMLkWLE2VKot/NKtKHVzyPmnRhjKkUl/V
5yqRecfZKM7Jvu4imWjfrYaInSaZsan1lCyyycaMgPBkbFPuAFbdIVkCgYEAtf2O
NqZOq8kVo6/y+9X02n+DTgpEGnWjht4rGnfvtXSMte8YnFUji89+gQRxnhVfZFB
xOw5UjGqkT/ILRzrpMcASbjSaRE8TSkGuacO3Nf62SmhsyF6/OK8DwqpijTksn39
M9R3eYBSv/0aCUXfoyrQ7392ojuU9mUnyNj9S8kCgYEAk1wZNUF4SQKglNZwbLa6
TGKqdz0WVLNGQB08vx8V7rl6pbygXI0F+dey8m8KnjuEl65FZgGyfhKi6y0rKymn
IFceV9vs89J66z24mwrROqXMXBd5rpp2aw6qi1DHYgX2dHRVmcSfmr5hCbr9lf0d
F8czDzCrjciRLVvEa2YmcVkCgYEAqQqtsCFI7bbH/NUP88/WsaXDhxeODq26iSmQ
C/PYNLBToJo3B5GRlsjK3pItGJbpgcsMtiddTvWHI4eFHjJAxtz5DwE3oBQ8WMeE
0alU6tlKzbAX3FV68a6BU89pj1B2JT1VIXECOL7bHi2wo3BdvCG1mmRhA2wof32S
KMM089ECgYAvRHEWpV/Xo1EtiwE6zoJIQ7QY3Wy7AgxuqA85y+3HT6WSQIXxAcy3
nlLJPPFsFFXG4WpY1weainoY1ZbSc2bQkD4oSL5ffFfy96CTVp/PPAvMFZH+/dm8
QbQD86uAFT11AW2d/2soikarpTCUMKdHCQtYNorgwLtoeKUtsVKzog==

-----END RSA PRIVATE KEY-----

Enter Message : HELLO

Original Message : HELLO

Encrypted Message (Cipher Text):

b'V\x87\xc9g\xe5\xed\x0f\x9e\xf3\x9a_\xcf\x83\x9b\xddvB\x93%\x83\x17\xc27\x89
\xa5\xed7\xde7e\xa3?\x9e\x1f#AN\x9d\x03\xdbB\x0fsH\x0b\xf4
q\x08\x1c\xfe\x09\x05:\xa4Mu\x82\xdf\x06\xe7\xe4\x0b\x80\x10G\xa5C\x16\x89\x
dbWx\x80u\x9b^\xba\x02\xce"\xe6;\xdc5\xf4*\x18\x94\x99\rKW\x0d\xff\xffT\x8e"\x
f0@I_\xfb\x1f\xb6\x15p\x83\x08\x05\x0e\x9d\xcc\x06\xfd\x19~\xdfbK\x9e\xa50\xe
7e\x1d\x0ei\xb7\x0eD\xf1\xe9\xa8#\x82V\xa8DtZ\xb4i\x09\x97&+\$_\xe2\x89S%\xd
5\x062&m`\xc1\x00\x1f\x8c\x88\xcf\x9da\x0d\x03\xdd\x922\x1ba\x15\x8f\xfa\xbf\
xe6\xa2.o\xf7\x14\x92\x0c\x9e\x0d\x01\xfe\x84nJ5\xa9\x9ab\xa8@\xdc\x0d\xddew\

x0c\xbe\x84_F\xdcf\xea\x9a\xb3\x83\xd1)\xaa\x0f\x06_j\x82\xfd\xba\x8d\x80\x9dtB
\xac/\x19d\xd5\xad<L J\x19\xe5\xf6\x92\xff\x99\xca\xa9\xcfu\xb5\xe4\x02\x8e\xa9'

Decrypted Message :

HELLO

RSA Encryption and Decryption Successful.

9. Step-by-Step Program Explanation

Step 1

Import RSA libraries.

Step 2

Generate a **2048-bit private key**.

Step 3

Generate the corresponding **public key**.

Step 4

Display both keys in PEM format.

Step 5

Accept a plaintext message from the user.

Step 6

Encrypt the plaintext using the **public key** and **OAEP padding** with SHA-256.

Step 7

Decrypt the ciphertext using the **private key**.

Step 8

Compare the decrypted message with the original plaintext to verify successful encryption and decryption.

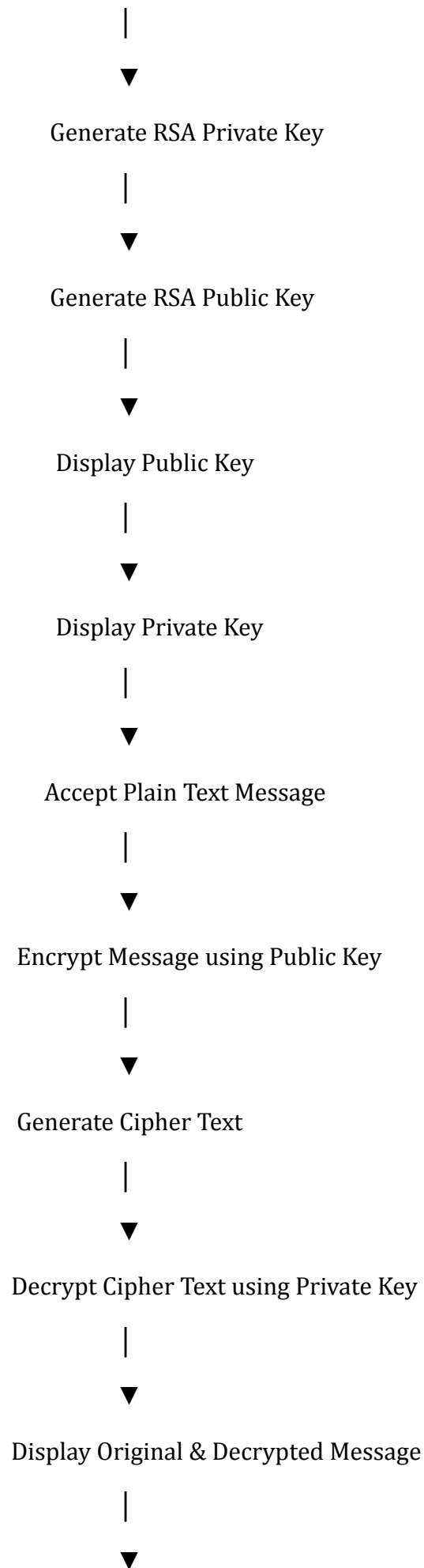
10. Detailed Execution Flow

START

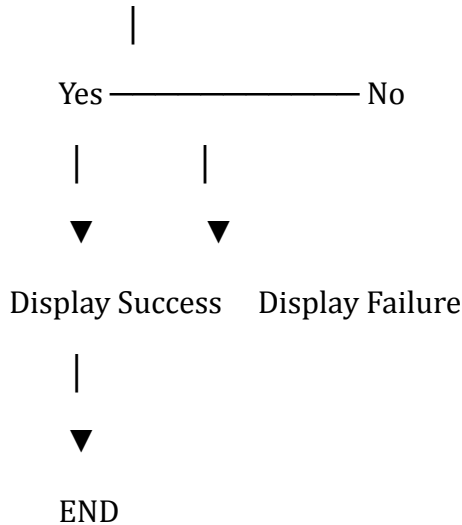
|



Import Required Libraries



Compare Original and Decrypted Text



11. Algorithm

1. Start the program.
2. Import the required RSA cryptography libraries.
3. Generate a 2048-bit RSA private key.
4. Generate the corresponding public key.
5. Display both keys in PEM format.
6. Read the plaintext message from the user.
7. Encrypt the message using the public key with OAEP and SHA-256.
8. Display the encrypted ciphertext.
9. Decrypt the ciphertext using the private key.
10. Display the decrypted plaintext.
11. Verify that the decrypted message matches the original message.
12. Display a success message if verification passes.
13. Stop the program.

12. Applications

- Secure Email Communication
- Digital Signatures
- HTTPS/SSL Certificates
- Internet Banking
- Blockchain Wallets
- Secure File Transfer
- VPN Authentication
- Government Secure Communication
- Cloud Security
- Software License Verification

13. Advantages

- High security based on asymmetric cryptography.
- Eliminates the need to share a secret key.
- Supports encryption, decryption, and digital signatures.

- Widely accepted and standardized.
- Suitable for secure key exchange.
- Resistant to brute-force attacks with large key sizes.

14. Result

The RSA algorithm was successfully implemented in Google Colab using Python. The program generated a secure RSA public-private key pair, encrypted the user-provided plaintext using the public key, and accurately decrypted the ciphertext using the corresponding private key. The decrypted message matched the original plaintext, confirming the correctness of the RSA implementation. This experiment demonstrates how asymmetric cryptography enables secure data transmission, authentication, and confidentiality in modern communication systems.